Luke Doughty

SOFT 260 Midterm Cheatsheet

Big O

\- Bachman Landau: $f(x) = 3x^3 + 2x^2 + x$

$\le 3x^3 + 2x^2 + x^2 = 3x^3 + 3x^2$ $x \ge 1$

$\le 3x^3 + 3x^3 = 6x^3$ $x \ge 1$

$f(x) \in O(6x^3)$

$f(x) \ge x^3 \therefore f(x) \in \Omega(x^3)$

$f(x) \in O(g(x))$ && $f(x) \in \Omega(g(x)) \therefore f(x) \in \Theta(g(x))$

Exhaustive Search

```
Input X: a list of numbers
let C ← ∅
for x ∈ X do
    if x == n then
        let C ← C ∪ {x}
    end
end
return C
```

\- **Limit Method** $f(x) = 3n;\ g(x) = \log_2 n$

① $\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0 \to O(g(n))$ ←

② $\lim_{n\to\infty} \frac{f(n)}{g(n)} > 0 \to \Theta(g(n))$

③ $\lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty \to \Omega(g(n))$

L'Hopital's rule $\lim_{n\to\infty} \frac{f(n)}{g(n)} = \lim_{n\to\infty} \frac{f'(n)}{g'(n)}$

Show that $\log_3 n \in O(n^{0.1})$

$\lim_{n\to\infty} \frac{\log_3 n}{n^{0.1}} = \lim_{n\to\infty} \frac{\frac{1}{n \ln(3)}}{0.1 n^{-0.9}} = \lim_{n\to\infty} \frac{1}{0.1 n^{0.1} \ln(3)} = 0$ ✓

☆ Dont use calculus unless needed. Use associative property

$\lim_{n\to\infty} x^2 + x$

$\lim_{n\to\infty} x^2 + \lim_{n\to\infty} x$

☆ $\Theta \subseteq O$

$\Theta \subseteq \Omega$

Add up all 1s - Divide & Conquer

```
function answer(A)
    Input A, a sorted list of zeros, ones
    if |A| = 0 then
        return 0
    end
    let m ← [|A|/2]
    if A[m] = 1 then
        return answer(A[0..m-1]) + (|A| - m)
    end
    return answer(A[m+1..|A|-1])
end
```

$\Theta(\log_2 n)$

Master Theorem

$T(n) = a\,T\left(\frac{n}{b}\right) + f(n)$ $f(n) = cn^k$

a ↑ # of subproblems

b ↑ Size of each subproblem

f(n) ← cost of work done outside of recursive call. Must be polynomial

$T(n) \in \Theta(n^k)$ if $a < b^k$

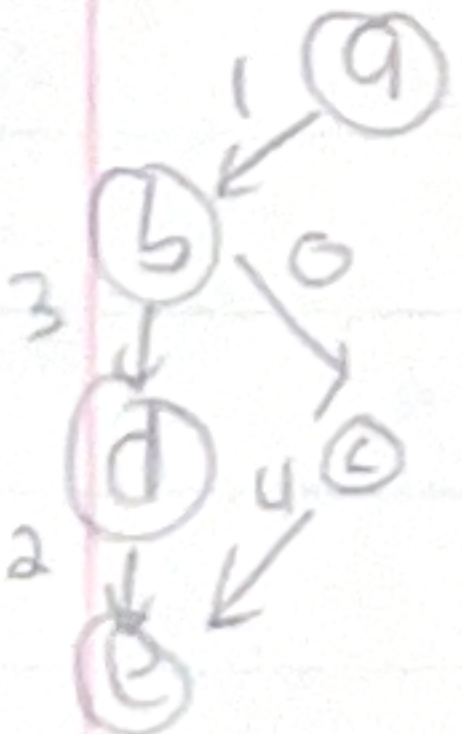
$T(n) \in \Theta(n^k \log_2 n)$ if $a = b^k$

$T(n) \in \Theta(n^{\log_b a})$ if $a > b^k$

$$T(n) = \begin{cases} T\left(\frac{n}{2}\right) + 1 & \text{if } n > 0 \\ 0 \quad n^0 & \text{otherwise} \end{cases}$$

$a = 0$ $k = 0$
$b = 2$

$1 = 2^0 \therefore \Theta(n^0 \log_2 n)$



BFS → queue → FIFO **DFS** → Stack → LIFO **Dijkstra's** → p-queue → FIFO*

BFS wl	BFS bp	DFS wl	DFS bp	Dijkstra's wl	Dijkstra's bp
(⊥, a)	a → (⊥, a)	(⊥, a)	a → (⊥, a)	(⊥, a, 0)	a → (⊥, a, 0)
(a, b)	b → (a, b)	(a, b)	b → (a, b)	(a, b, 1)	b → (a, b, 1)
(b, c)	c → (b, c)	(b, c)	d → (b, d)	(b, c, 1)	c → (b, c, 1)
(b, d)	d → (b, d)	(b, d)	e → (d, e)	(b, d, 4)	d → (b, d, 4)
(c, e)	e → (c, e)	(d, e)	c → (b, c)	(c, e, 5)	e → (c, e, 5)
(d, e)		(c, e)		(d, e, 6)	

A* adds heuristic like euclidian distance to target

$\sum_{i=0}^{n-1} 1 = n$

$\sum_{i=0}^{n-1} \sum_{j=0}^{i-1} 1 = \sum_{i=0}^{n-1} i = \frac{i(i-1)}{2}\Big|_0^n = \frac{n(n-1)}{2}$

Sorting $\theta(m\log(m))$

$\theta(n^3)$

Kruskal's Method

- sort edges by weight
- add edge with smallest weight
  - ↳ must not form a cycle

Prim's Algorithm

- start with arbitrary vertex
- build subtrees adding vertexes of smallest weight first

Asymptotic Complexity

Ⓒ prove order of $\theta(1), \theta(\log n), \theta(n), \theta(n^2)$ from slowest to fastest

$$\lim_{n\to\infty} \frac{1}{\log n} = 0$$

$$\lim_{n\to\infty} \frac{\log n}{n} = 0 \longleftarrow = \lim_{n\to\infty} \frac{\frac{1}{n\ln(2)}}{n} = \lim_{n\to\infty} \frac{1}{n^2\ln(2)} = 0$$

$$\lim_{n\to\infty} \frac{n}{n^2} = \frac{1}{n} = 0$$

Types of Exhaustive Searches

- Find all matching
- Find one matching
- Find all maximizing/minimizing (optimal value)

Luke Doughty SOFT 260 Final Pt. 2 Cheat Sheet

Dynamic Programming - Good for problems with overlapping subproblems.

Bottom up Fibbonacci

```
 0 1 ...  n
[1 1 ...   ] = f
```

```
BU_fib(n)
    f[0] = 1
    f[1] = 1
    for i = 2,...,n
        f[i] = f[i-1] + f[i-2]
    return f[n]
```

Top down Fibbonacci

[1 1 .... n] = array

```
TD_fib(n)
    if n==1 or n==0 : return 1
    if array[n] != 0    #not precomputed        novel subproblem
        return array[n]
    array[n] = TD_fib(n-1, array) + TD_fib(n-2, array)
    return array[n]
```

memoization: store results of expensive function calls. Returns same result when same problem occurs.

Knapsack Problems

Unbounded → Unlimited of each item → $\frac{\$}{w}$

max weight (W) = 8

Bounded

Items	$	W
A	2	1
B	4	3
C	7	5
D	10	7

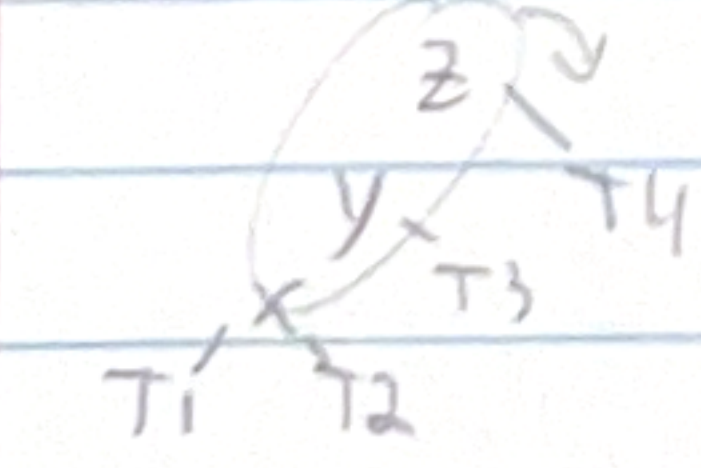
w+1

	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
A	0	2	2	2	2	2	2	2	2
B	0	2	2	4	6	6	6	6	6
C	0	2	2	4	6	7	9	9	11
D	0	2	2	4	6	7	9	10	12

profit: 12
items A and D

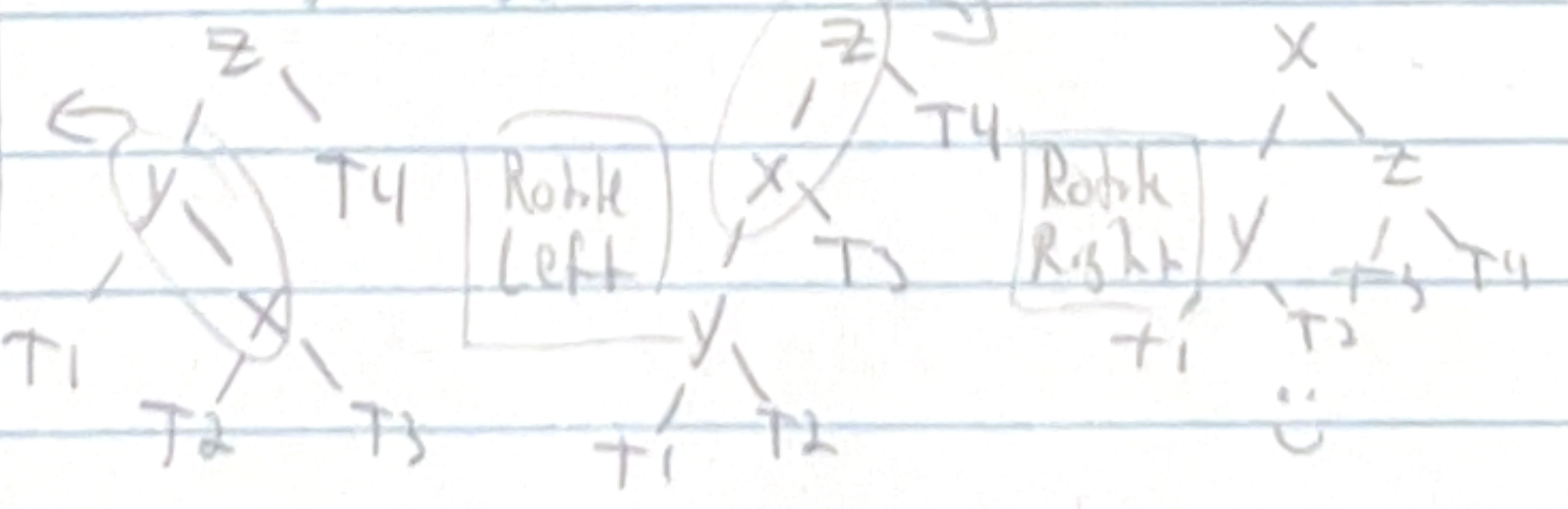
AVL Tree - self balancing Binary Tree - difference in depth of left and right subtrees cannot be more than one for all nodes

Left Left case



Needs a right rotation

Left Right Case



Balance Factor for any node

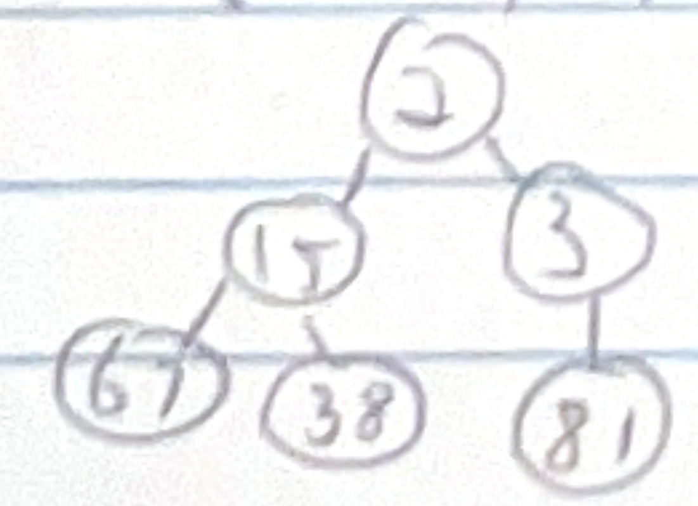
$b(x) = height(left) - height(right)$

height of an empty tree (-1)

$b(x) \in \{-1, 0, 1\}$ or else rebalance

Heap - like a priority queue but a tree

given [15, 67, 3, 81, 2, 38]



is one of many valid min heaps

*all that matters is that the larger numbers are deeper in the tree.

A max tree is the opposite

Unary Tree ⟺ Linked List

Binary Search Tree: each node has at most two children. Left is smaller, right is larger than the current node.

Hashing - Map or dictionary - efficiently pulls from a collection

$\hat{f} = f(x) \% b$ Load Factor: $\frac{\#\text{ of entries}}{\#\text{ of buckets}} \in [\frac{2}{3}, \frac{3}{4}]$

"Store 5 pairs of ints in a hash table"

$f(P) = 3P[0] + P[1] \leftarrow$ 2 buckets load factor: $\frac{5}{2} \leftarrow$ need to resize

bad case for 10 buckets: (10,10) (20,20) (30,30) . . .

or (0,0) (0,10), (0,20), . . .

best case for 10 buckets: (0,1) (0,2) (0,3) . . .

good with 10, bad with 2: (0,2), (0,4), (0,6) . . .

DAG - directed acyclic graph

vertices ← state

edges ← action